

Module 10 - chapter 23

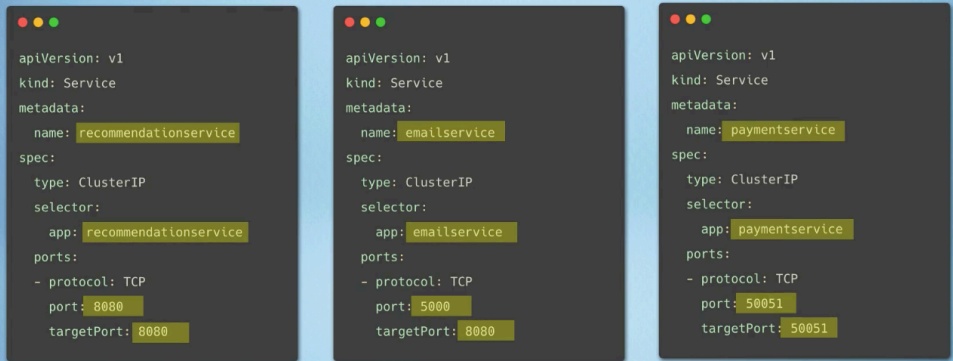
Demo project: Create Helm Chart for Microservices

The link to the Microservices Helm Chart project can be found here: <https://gitlab.com/twn-devops-bootcamp/latest/10-kubernetes/helm-chart-microservices>

We have 10 similar deployments and service configuration in the config.yaml file. Almost the same attributes with different values:

Introduction

10 Similar Deployment and Service Configurations



```
apiVersion: v1
kind: Service
metadata:
  name: recommendation
spec:
  type: ClusterIP
  selector:
    app: recommendation
  ports:
    - protocol: TCP
      port: 8080
      targetPort: 8080
```

```
apiVersion: v1
kind: Service
metadata:
  name: email
spec:
  type: ClusterIP
  selector:
    app: email
  ports:
    - protocol: TCP
      port: 5000
      targetPort: 8080
```

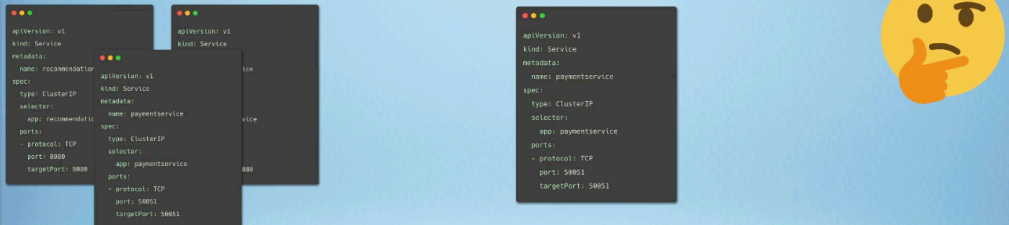
```
apiVersion: v1
kind: Service
metadata:
  name: payment
spec:
  type: ClusterIP
  selector:
    app: payment
  ports:
    - protocol: TCP
      port: 50051
      targetPort: 50051
```

Copyright © TechWorld with Nana, a brand of nnSoftware GmbH. All rights reserved

So we can take advantage of the similarities and create One blueprint for deployment and service for all MS and then set the values for individual MS

Introduction

10 Similar Deployment and Service Configurations



```
apiVersion: v1
kind: Service
metadata:
  name: recommendation
spec:
  type: ClusterIP
  selector:
    app: recommendation
  ports:
    - protocol: TCP
      port: 8080
      targetPort: 8080
```

```
apiVersion: v1
kind: Service
metadata:
  name: payment
spec:
  type: ClusterIP
  selector:
    app: payment
  ports:
    - protocol: TCP
      port: 50051
      targetPort: 50051
```

- ▶ 1 Blueprint for Deployment and Service for all microservices
- ▶ Set values for individual microservices

Copyright © TechWorld with Nana, a brand of nnSoftware GmbH. All rights reserved

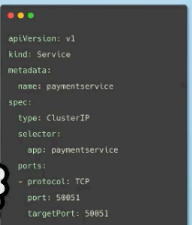
And as we saw in Module 10 - chapter 15 we can use helm chart to do exactly that -> create a blueprint!

Introduction

10 Similar Deployment and Service Configurations



Helm Charts



```
apiVersion: v1
kind: Service
metadata:
  name: payment-service
spec:
  type: ClusterIP
  selector:
    app: payment-service
  ports:
    - protocol: TCP
      port: 50051
      targetPort: 50051
```

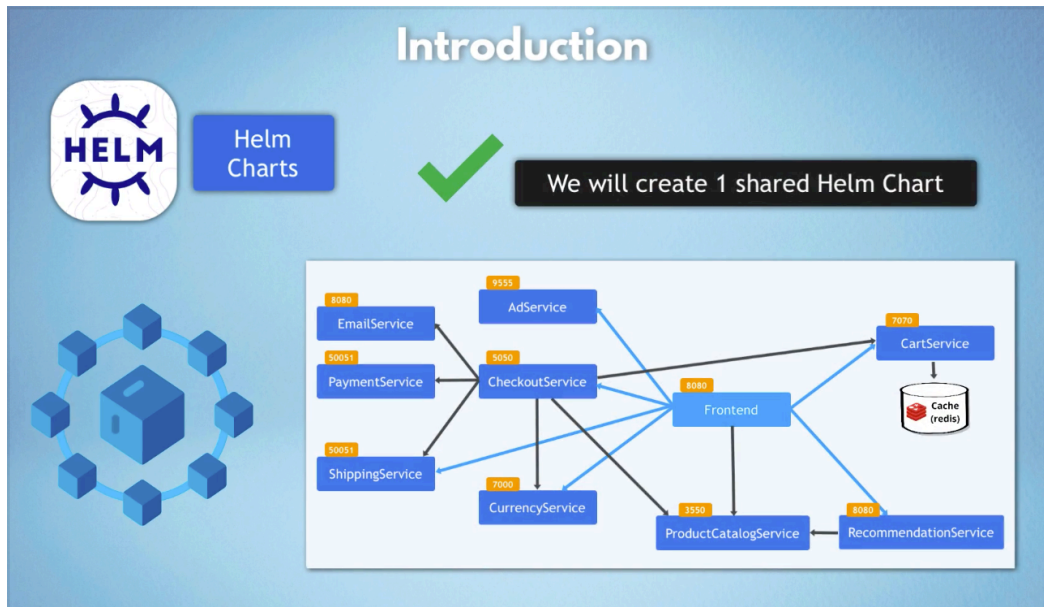
- ▶ 1 Blueprint for Deployment and Service for all microservices
- ▶ Set values for individual microservices

Microservices are one of the best use cases of Helm charts because one of helm charts advantages is to define reusable k8s configuration -> which can be found in microservices.

We can create a helm chart:

- 1 helm chart per MS -> if the config are very different
- 1 Shared helm chart for all MSs
- Combination of both:
 - Shared chart for similar apps
 - Separate charts for completely different apps

For our DEMO the config is almost the same so we will create 1 shared Helm Chart

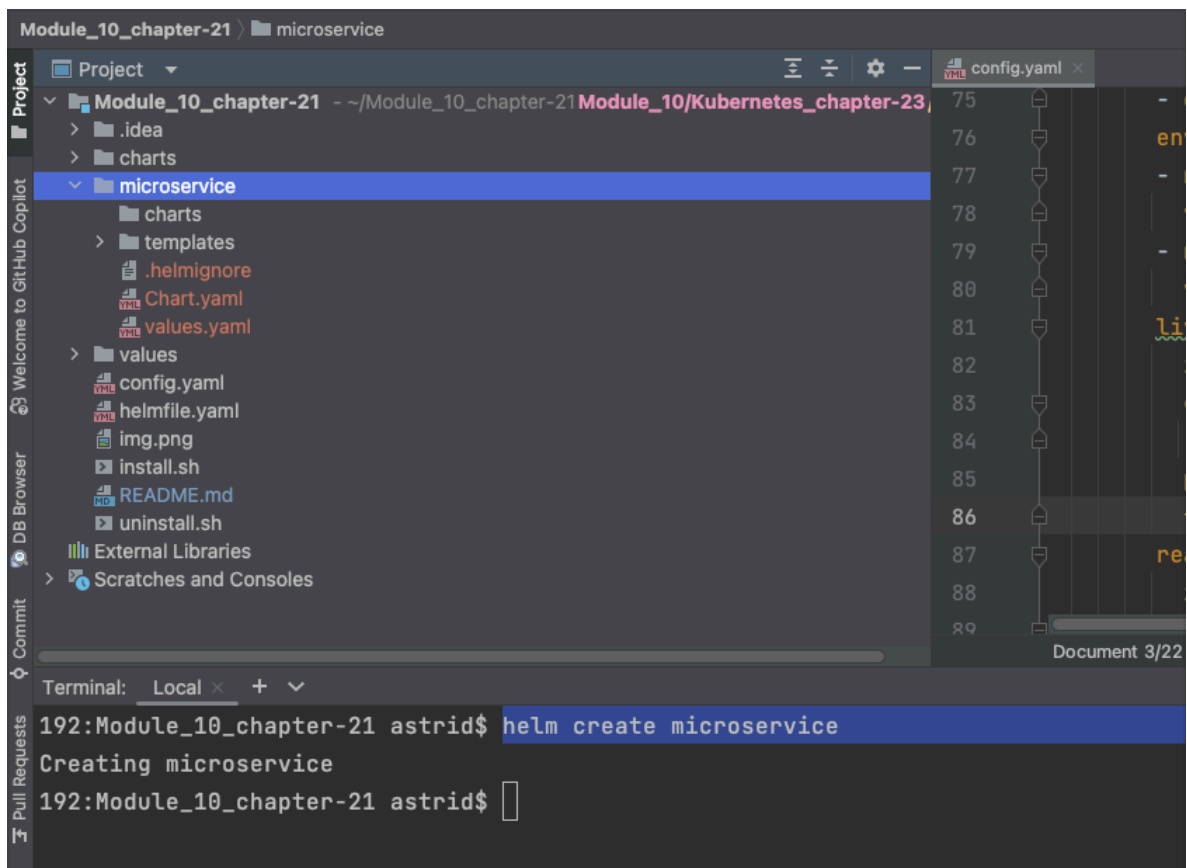


Basic structure of Helm chart

Let's create helm chart in our project:

-> `helm create microservice`

Microservice -> is the name we are giving to our helm chart

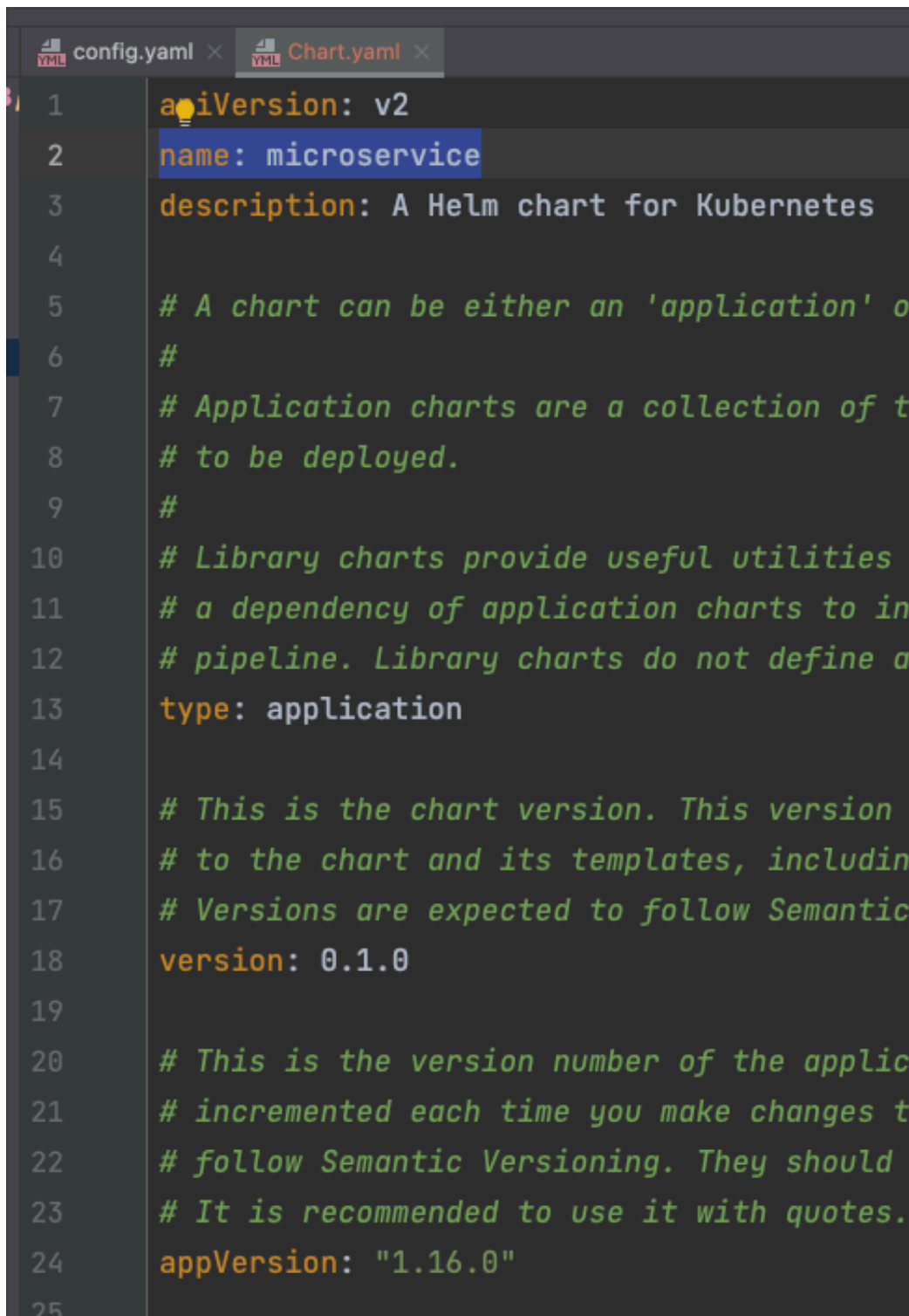


```
Module_10_chapter-21 > microservice
Project
  Module_10_chapter-21 - ~/Module_10_chapter-21
    .idea
    charts
    microservice
      charts
      templates
      .helmignore
      Chart.yaml
      values.yaml
      values
        config.yaml
        helmfile.yaml
        img.png
        install.sh
        README.md
        uninstall.sh
      External Libraries
      Scratches and Consoles

Terminal: Local x + v
192:Module_10_chapter-21 astrid$ helm create microservice
Creating microservice
192:Module_10_chapter-21 astrid$
```

So now we have:

- a 'microservice' folder -> that is our chart
- Chart.yaml -> describes the metadata about the chart

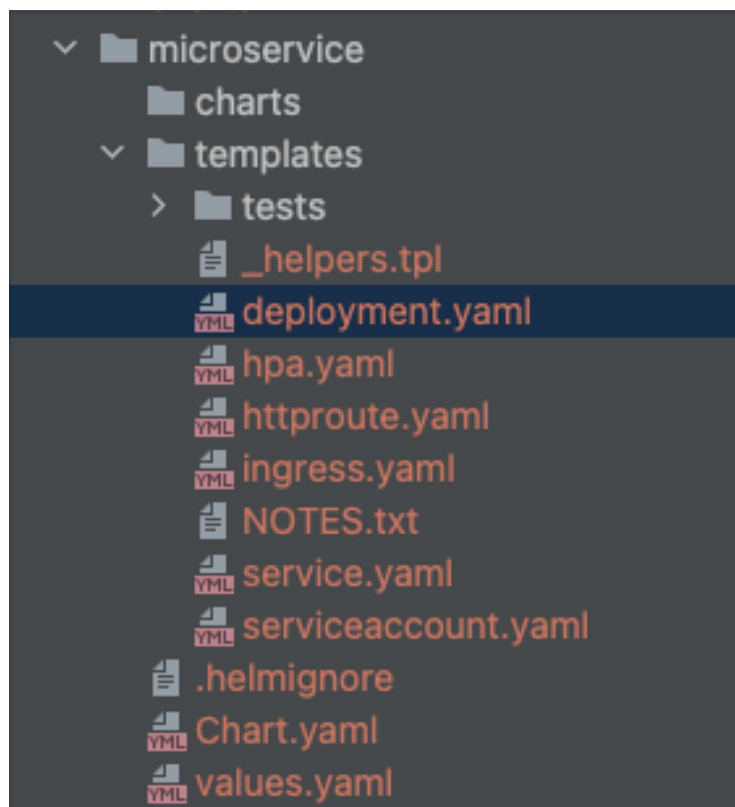


```
1 apiVersion: v2
2 name: microservice
3 description: A Helm chart for Kubernetes
4
5 # A chart can be either an 'application' or 'library' chart.
6 #
7 # Application charts are a collection of templates that can be deployed.
8 # to be deployed.
9 #
10 # Library charts provide useful utilities and are used as a dependency of application charts to inject
11 # a dependency of application charts to inject into its templating.
12 # pipeline. Library charts do not define any templates.
13 type: application
14
15 # This is the chart version. This version must match the version of the chart and its templates, including
16 # to the chart and its templates, including
17 # Versions are expected to follow Semantic Versioning (https://semver.org/)
18 version: 0.1.0
19
20 # This is the version number of the application being deployed. This version number must be incremented each time you make changes to the application.
21 # incremented each time you make changes to the application.
22 # follow Semantic Versioning. They should be unique and immutable.
23 # It is recommended to use it with quotes.
24 appVersion: "1.16.0"
25
```

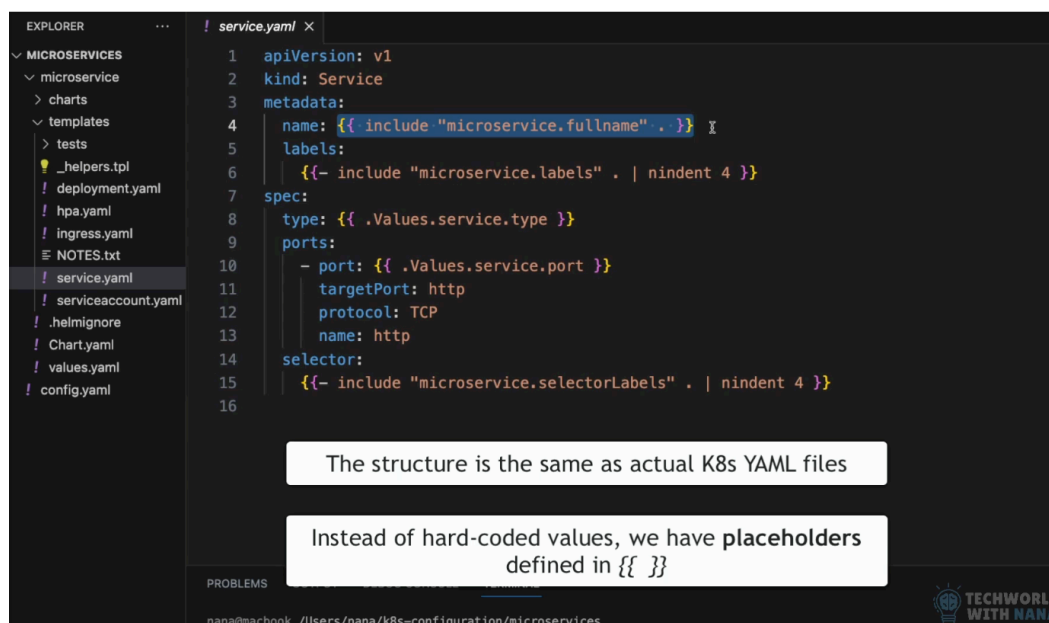
- Charts folder -> holds any dependency charts that this 'microservice' chart might have. For our DEMO we don't have chart dependencies, so the folder will remain empty.
- .helmignore -> the files we add to this .helmignore are the ones we don't want to be included in our helm chart when building the package archive from this 'microservice' helm chart. In this demo we won't build a package so we can ignore this file but is good to know that helm charts can be built and distributed just like any other artifact

```
config.yaml x .helmignore x
3 Plugins supporting *.helmignore files found.
1 # Patterns to ignore when building packages.
2 # This supports shell glob matching, relative path matching, and
3 # negation (prefixed with !). Only one pattern per line.
4 .DS_Store
5 # Common VCS dirs
6 .git/
7 .gitignore
8 .bzzr/
9 .bzrignore
10 .hg/
11 .hgignore
12 .svn/
13 # Common backup files
14 *.swp
15 *.bak
16 *.tmp
17 *.orig
18 *~
19 # Various IDEs
20 .project
21 .idea/
22 *.tmproj
23 .vscode/
```

- Templates folder -> core of the helm chart, where the actual k8s YAML files (template files) are created. So here we will create the k8s YAML files and we can see that the files inside the folder have similar names to the K8s components like deployment or Ingress, etc:



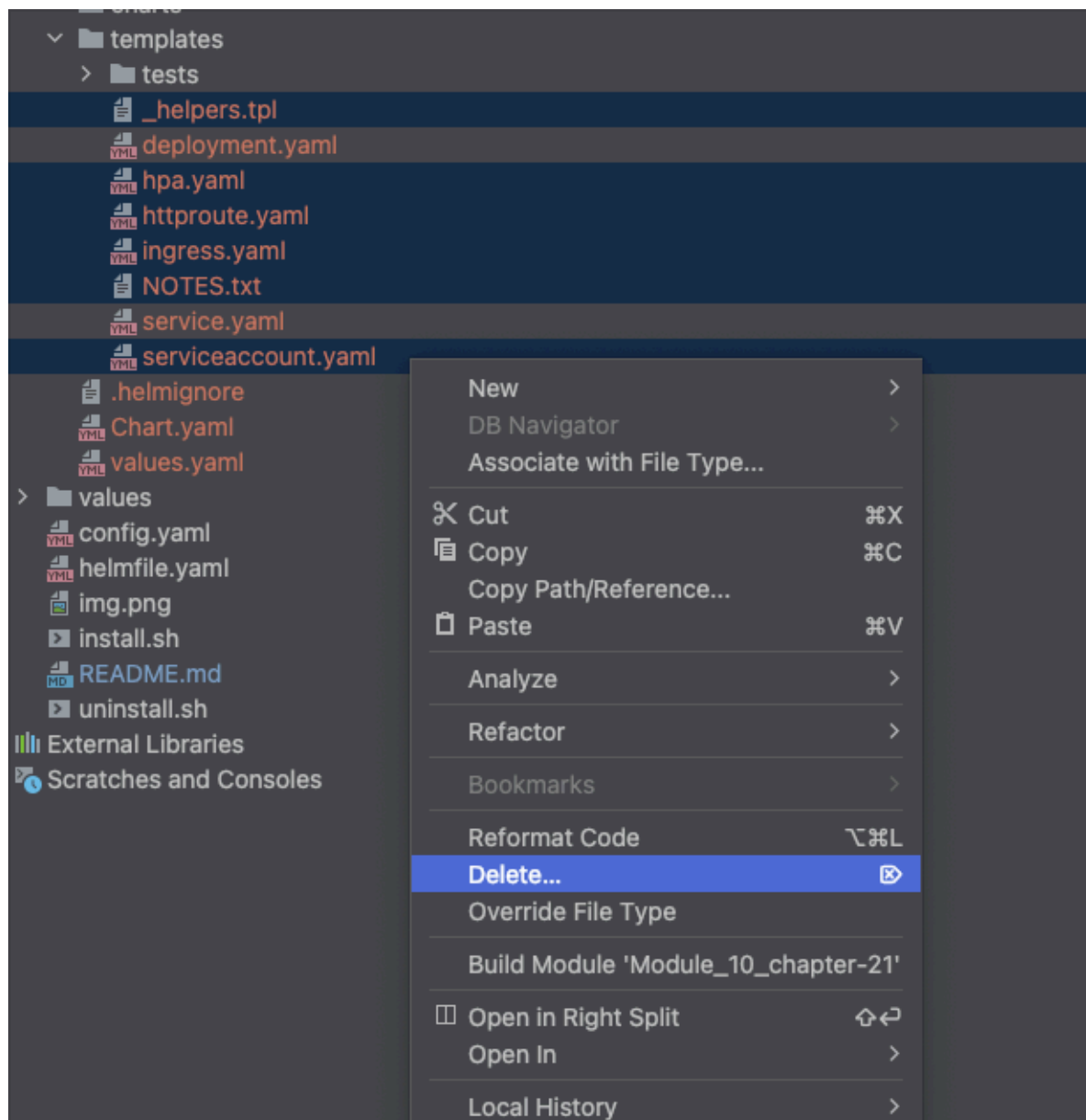
If we take a look at one of the yaml files like service.yaml we can see that the structure is the same as the actual K8s YAML file BUT the values are not hard-coded -> we have placeholders defined in {{ }}



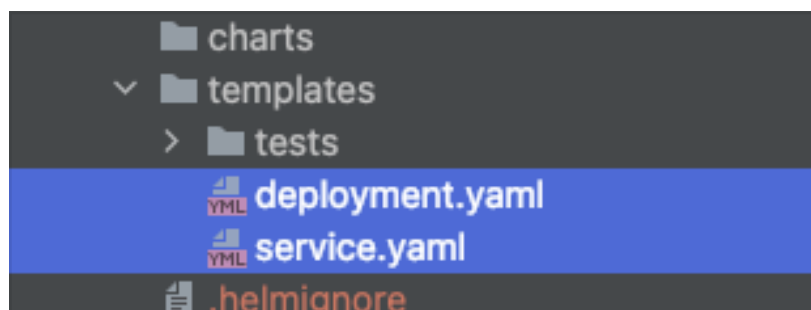
Later Helm will replace the placeholder with the actual values ❤️

So service.yaml is actually a template!

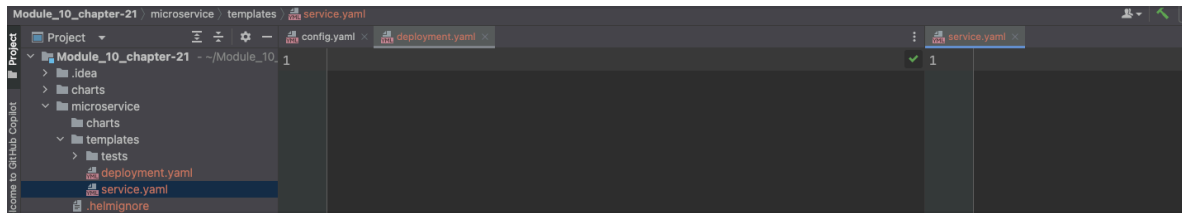
For the demo we will write our own templates and we will remove the autogenerated template files:



We are left with two templates: deployment and service:



And we remove the contents of those templates, so we can write it ourselves:



- values.yaml file -> is also part of the 'microservice' helm chart and is where the actual values for the template files are set.
- We will remove the content of values.yaml as well.

Create a basic template file for deployment:

```
deployment.yaml x values.yaml x
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: xxxx
5  spec:
6    replicas: 2
7    selector:
8      matchLabels:
9        app: xxxx
10   template:
11     metadata:
12       labels:
13         app: xxxx
14   spec:
15     containers:
16       - name: xxxx
17         image: xxxx
18         ports:
19           - containerPort: xxxx
```

And for service:

```
Module_10_chapter-21 > microservice > templates > service.yaml
deployment.yaml x service.yaml x
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: xxxx
5  spec:
6    type: ClusterIP
7    selector:
8      app: xxxx
9    ports:
10     - protocol: TCP
11       port: xxxx
12       targetPort: xxxx
```

How to write a placeholder variable

-> `{{ .Values.varName }}`

Values -> is an Object the 'Values' Object.

- by default is empty
- but when values are defined (3 ways to do this, one is using the values.yaml file) they get injected into the placeholder variable
- Values is one of the helm built-in Objects but there are other: Release, Files, etc (the objects start with a capital letter)

varName -> name of the variable, we decide that.

- variable names can be whatever we want but they have a standard a naming convention: start with lowercase and use camelcase format like 'varName'

The screenshot shows a code editor with three tabs: `deployment.yaml`, `values.yaml`, and `service.yaml`. The `deployment.yaml` file contains the following YAML:

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: {{ .Values.varName }}
5 spec:
6   selector:
7     matchLabels:
8       app: xxxx
9   template:
10     metadata:
11       labels:
12         app: xxxx
13     spec:
14       containers:
15       - name: xxxx
16         image: xxxx
17         ports:
18         - containerPort: xxxx
```

A blue overlay titled **"Values" Object** is positioned on the right side of the editor. It contains the following text:

- ▶ A built-in object
- ▶ By default, *Values* is empty
- ▶ Values are passed into template, from **3 sources**:

Below the list, three white boxes with black text provide the sources:

- the *values.yaml* file in the chart
- user-supplied file passed with `-f` flag
- parameter passed with `--set` flag

The RLD INA logo is visible in the bottom right corner of the overlay.

note: User-supplied file -> means a custom values file 🙌 different to the default values.yaml file

Let's create our first placeholder variable:

The screenshot shows a code editor with two tabs: `deployment.yaml` and `service.yaml`. The `deployment.yaml` file contains the following YAML:

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: { { .Values.appName } }
5 spec:
6   replicas: 2
```

appName -> application name like 'emailservice' etc

And we can use that placeholder for other attributes where it is required:

```
deployment.yaml
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: { { .Values.appName } }
5  spec:
6    selector:
7      matchLabels:
8        app: { { .Values.appName } }
9    template:
10     metadata:
11       labels:
12         app: { { .Values.appName } }
13     spec:
14       containers:
15         - name: { { .Values.appName } }
16           image: XXXX
17           ports:
18             - containerPort: XXXX

service.yaml
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: { { .Values.appName } }
5  spec:
6    type: ClusterIP
7    selector:
8      app: { { .Values.appName } }
9    ports:
10     - protocol: TCP
11       port: XXXX
12       targetPort: XXXX
```

Now let's work on the container port:

```
deployment.yaml
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: { { .Values.appName } }
5  spec:
6    selector:
7      matchLabels:
8        app: { { .Values.appName } }
9    template:
10     metadata:
11       labels:
12         app: { { .Values.appName } }
13     spec:
14       containers:
15         - name: { { .Values.appName } }
16           image: XXXX
17           ports:
18             - containerPort: { { .Values.containerPort } }

service.yaml
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: { { .Values.appName } }
5  spec:
6    type: ClusterIP
7    selector:
8      app: { { .Values.appName } }
9    ports:
10     - protocol: TCP
11       port: XXXX
12       targetPort: { { .Values.containerPort } }
```

Next, service port:

```
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: { { .Values.appName } }
5  spec:
6    type: ClusterIP
7    selector:
8      app: { { .Values.appName } }
9    ports:
10     - protocol: TCP
11       port: { { .Values.servicePort } }
12       targetPort: { { .Values.containerPort } }
```

And the service type:

```
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: { { .Values.appName } }
5  spec:
6    type: { { .Values.serviceType } }
7    selector:
8      app: { { .Values.appName } }
9    ports:
10     - protocol: TCP
11       port: { { .Values.servicePort } }
12       targetPort: { { .Values.containerPort } }
```

We also need to configure the image for the deployment template:
Image = appName + appVersion -> "appName : appVersion"

```
deployment.yaml
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: { { .Values.appName } }
5  spec:
6    selector:
7      matchLabels:
8        app: { { .Values.appName } }
9    template:
10     metadata:
11       labels:
12         app: { { .Values.appName } }
13     spec:
14       containers:
15         - name: { { .Values.appName } }
16           image: "{ { .Values.appImage } }:{ { .Values.appVersion } }"
17           ports:
18             - containerPort: { { .Values.containerPort } }
```

Another configurable attribute that we want is the replica count:

```
deployment.yaml
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: { { .Values.appName } }
5  spec:
6    replicas: { { .Values.appReplicas } }
7    selector:
8      matchLabels:
9        app: { { .Values.appName } }
10   template:
11     metadata:
12       labels:
13         app: { { .Values.appName } }
14     spec:
15       containers:
16         - name: { { .Values.appName } }
17           image: "{ { .Values.appImage } }:{ { .Values.appVersion } }"
18           ports:
19             - containerPort: { { .Values.containerPort } }
```

In helm we can also use hierarchy when defining the variables, so instead of 'appName' we can also do 'app.name' so when we set the values instead of FLAT values:

```
deployment.yaml  values.yaml
1  apiVersion: apps/v1      1  appName: myapp
2  kind: Deployment        2  appReplicas: 1
3  metadata:
4    name: { { .Values.appName } }
5  spec:
6    replicas: { { .Values.appReplicas } }
7    selector:
```

We do it like this, use Nested or hierarchical values:

```
deployment.yaml  values.yaml
1  apiVersion: apps/v1      1  app:
2  kind: Deployment        2    name: myapp
3  metadata:              3    replicas: 1
4    name: { { .Values.app.Name } }
5  spec:
6    replicas: { { .Values.app.Replicas } }
7    selector:
```

Helm recommends using Flat values:

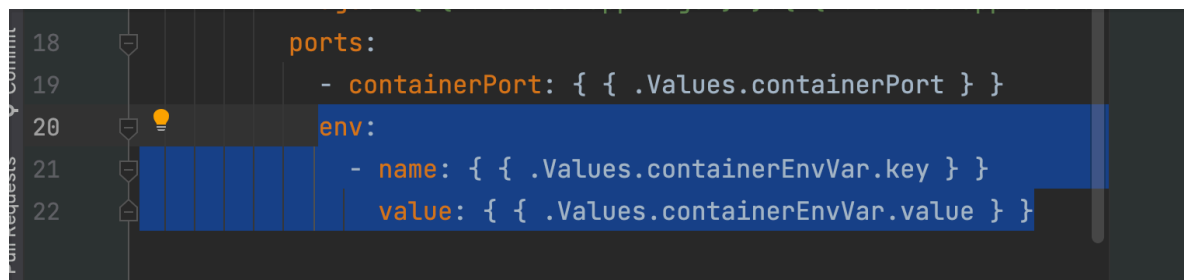
Flat or Nested Values

- ▶ Values may be flat or nested deeply
- ▶ Best practice is to use flat structure, which is simpler

Dynamic Environment Variables

We are missing to define our env vars, all of the MS have at least one env var.

If we only have one env var in all the MSs (every single MS has only one env var)



```
18     ports:
19       - containerPort: { { .Values.containerPort } }
20     env:
21       - name: { { .Values.containerEnvVar.key } }
22         value: { { .Values.containerEnvVar.value } }
```

But we have multiple env vars in some MS and only one env var in other MS, so we want this to be configurable.

So we have a list of env vars and helm has something called 'RANGE' that we can use like we do when writing loops:

"range"

- ▶ Provides a "for each"-style loop
- ▶ To iterate through or "range over" a list

So we create the variable for the list and we use range:

-> `{{-range .Values.containerEnvVars}}`

range -> will loop through the list

containerEnvVars -> list variable, holding all the env vars

```
19     - containerPort: { { .Values.containerPort } }
20     env:
21       {{-range .Values.containerEnvVars}}
22       - name: { { .key } }
23         value: { { .value } }
```

One last thing, the actual value for the 'value' attribute in the 'env' section has to be a string, see line 23 📌

```
Module_10_chapter-21 > config.yaml
deployment.yaml x config.yaml x
10     app: emailservice
11     template:
12       metadata:
13         labels:
14           app: emailservice
15       spec:
16         containers:
17         - name: service
18           image: gcr.io/google-samples/microservices
19           ports:
20             - containerPort: 8080
21           env:
22             - name: PORT
23               value: "8080"
24           livenessProbe:
25             initialDelaySeconds: 10
26           grpc:
27             port: 8080
```

So we need to do that in our template. For that we use a built-in function called 'quote' and we also use pipetting (|)

Pipelines

- ▶ Same concept as UNIX
- ▶ Tool for chaining together template commands

-> { { .value | quote } }

```
22     - name: { { .key } }  
23     value: { { .value | quote } }
```

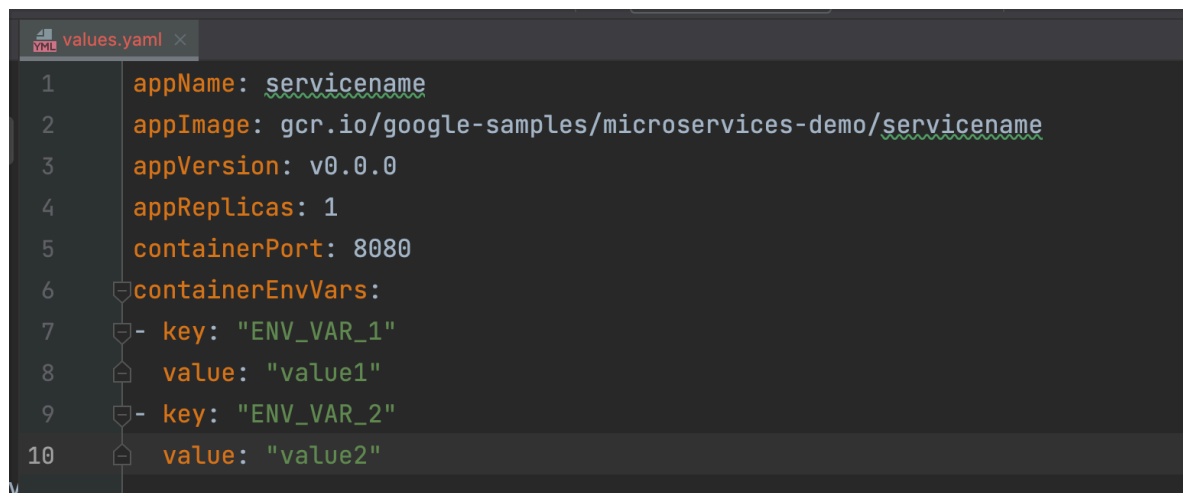
And finally we close the loop:

-> {{- end }}

```
19     - containerPort: { { .Values.containerPort } }  
20     env:  
21     {{-range .Values.containerEnvVars}}  
22     - name: { { .key } }  
23     value: { { .value | quote } }  
24     {{- end }}
```

Set values

We will use the values.yaml to set **default** values later we will **override** those values depending on the needs of each MS:



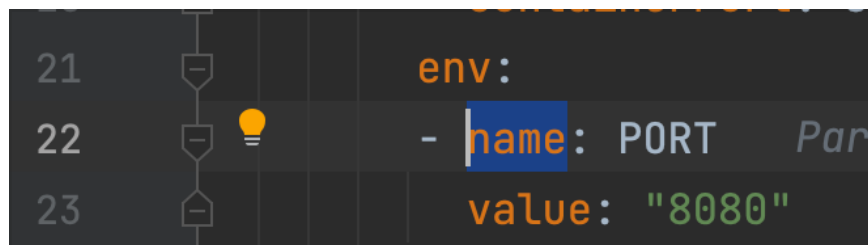
```

1  appName: servicename
2  appImage: gcr.io/google-samples/microservices-demo/servicename
3  appVersion: v0.0.0
4  appReplicas: 1
5  containerPort: 8080
6  containerEnvVars:
7    - key: "ENV_VAR_1"
8      value: "value1"
9    - key: "ENV_VAR_2"
10     value: "value2"

```

containerEnvVars is a list of key:value pairs and we see above how to set the default values for each env var.

In our previous demo we didn't use the keyword 'key' we used 'name':

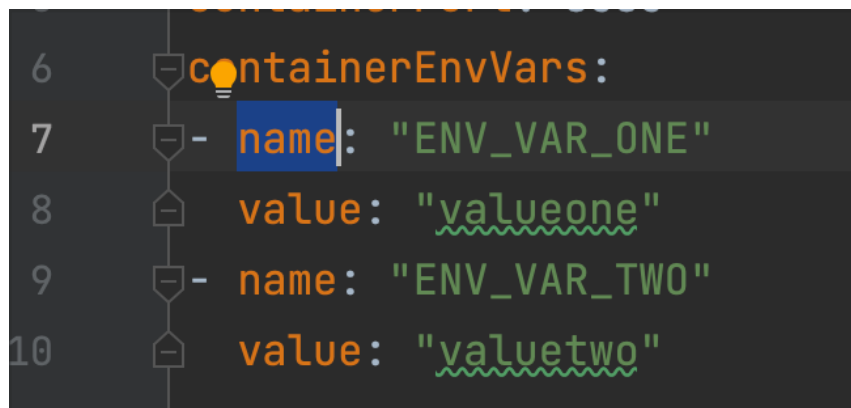


```

21  env:
22    - name: PORT
23      value: "8080"

```

So let's use that keyword instead:



```

6  containerEnvVars:
7    - name: "ENV_VAR_ONE"
8      value: "valueone"
9    - name: "ENV_VAR_TWO"
10     value: "valuetwo"

```

We set default values for the placeholders vars in deployment, next let's add the placeholders vars from the service template to the same values.yaml (some of the service.yaml placeholders are already in the values.yaml because they are also used by the deployment.yaml file):

```
1  appName: servicename
2  appImage: gcr.io/google-samples/microservices-demo/servicename
3  appVersion: v0.0.0
4  appReplicas: 1
5  containerPort: 8080
6  containerEnvVars:
7  - name: "ENV_VAR_ONE"
8    value: "valueone"
9  - name: "ENV_VAR_TWO"
10   value: "valuetwo"
11
12  servicePort: 8080
13  serviceType: ClusterIP
```

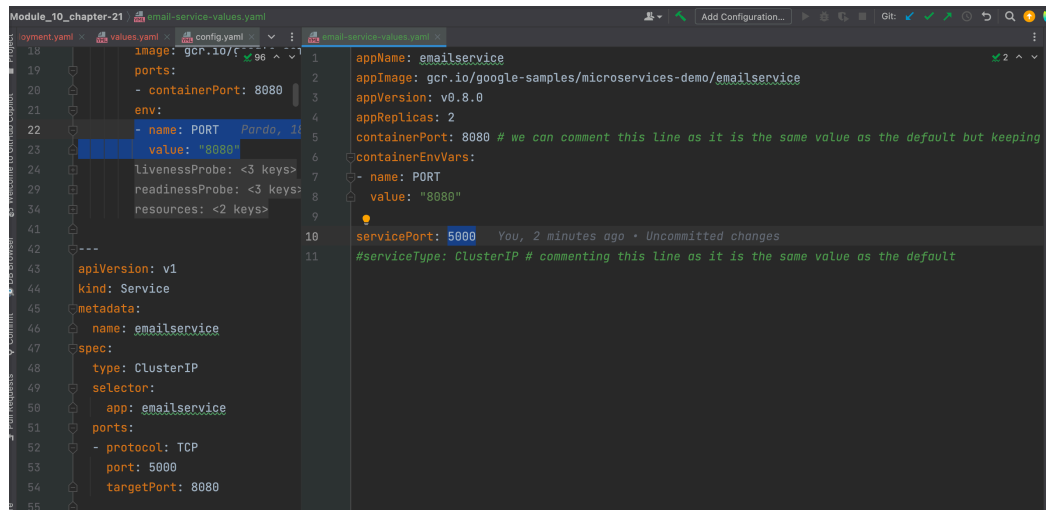
All set, BUT the value are default, we need to override them with the actual values per MS and for that we create our own CUSTOM values file:

- Create a file outside the microservice helm chart folder called 'email-service-values.yaml' and paste the default values from the values.yaml file:

The screenshot shows a VS Code editor with two files open: 'values.yaml' and 'email-service-values.yaml'. The 'email-service-values.yaml' file is being created with the same content as 'values.yaml'. The file explorer on the left shows the project structure, including the 'email-service-values.yaml' file. The status bar at the bottom indicates that the file is 'Uncommitted'.

```
1  appName: servicename
2  appImage: gcr.io/google-samples/microservices-demo/servicename
3  appVersion: v0.0.0
4  appReplicas: 1
5  containerPort: 8080
6  containerEnvVars:
7  - name: "ENV_VAR_ONE"
8    value: "valueone"
9  - name: "ENV_VAR_TWO"
10   value: "valuetwo"
11
12  servicePort: 8080
13  serviceType: ClusterIP
```

Now, let's substitute the default values for the actual ones in the email-service-values.yaml file:



Check we are producing valid k8s yaml files

Use helm commands

helm template =
render chart templates locally and display the output

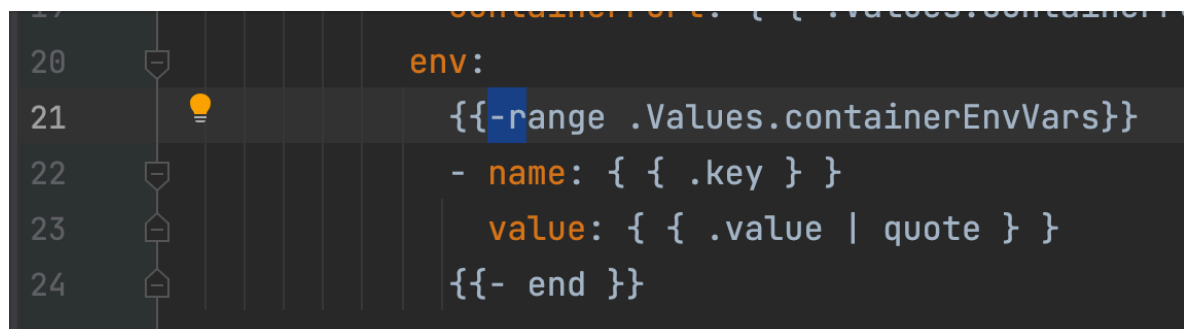
-> `helm template -f email-service-values.yaml microservice`

microservice -> name of helm chart

```
192:Module_10_chapter-21 astrid$ helm template -f email-service-values.yaml microservice
Error: parse error at (microservice/templates/deployment.yaml:21): "-r"

Use --debug flag to render out invalid YAML
192:Module_10_chapter-21 astrid$
```

We got an error, we are missing a space in line 21 between '-' and 'r'



Fix it (there were other issues I fixed like indentation and I removed space between curly braces and in line 22 use '.name' instead of '.key')

```

18     ports:
19       - containerPort: {{ .Values.containerPort }}
20     env:
21     {{- range .Values.containerEnvVars}}
22       - name: {{ .name }}
23         value: {{ .value | quote }}
24     {{- end }}

```

And I fixed the service as well:

```

1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: {{ .Values.appName }}
5  spec:
6    type: {{ .Values.serviceType }}
7    selector:
8      app: {{ .Values.appName }}
9    ports:
10     - protocol: TCP
11       port: {{ .Values.servicePort }}
12       targetPort: {{ .Values.containerPort }}

```

Then I tried again and it worked 🥳

```

D Use --debug flag to render out invalid YAML
192:Module_10_chapter-21 astrid$ helm template -f email-service-values.yaml microservice
---
# Source: microservice/templates/service.yaml
apiVersion: v1
kind: Service
metadata:
  name: emailservice
spec:
  type: ClusterIP
  selector:
    app: emailservice
  ports:
    - protocol: TCP
      port: 5000
      targetPort: 8080
---
# Source: microservice/templates/deployment.yaml

```

```

# Source: microservice/templates/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: emailservice
spec:
  replicas: 2
  selector:
    matchLabels:
      app: emailservice
  template:
    metadata:
      labels:
        app: emailservice
    spec:
      containers:
        - name: emailservice
          image: "gcr.io/google-samples/microservices-demo/emailservice:v0.8.0"
          ports:
            - containerPort: 8080
          env:
            - name:
              value: "8080"
192:Module_10_chapter-21 astrid$

```

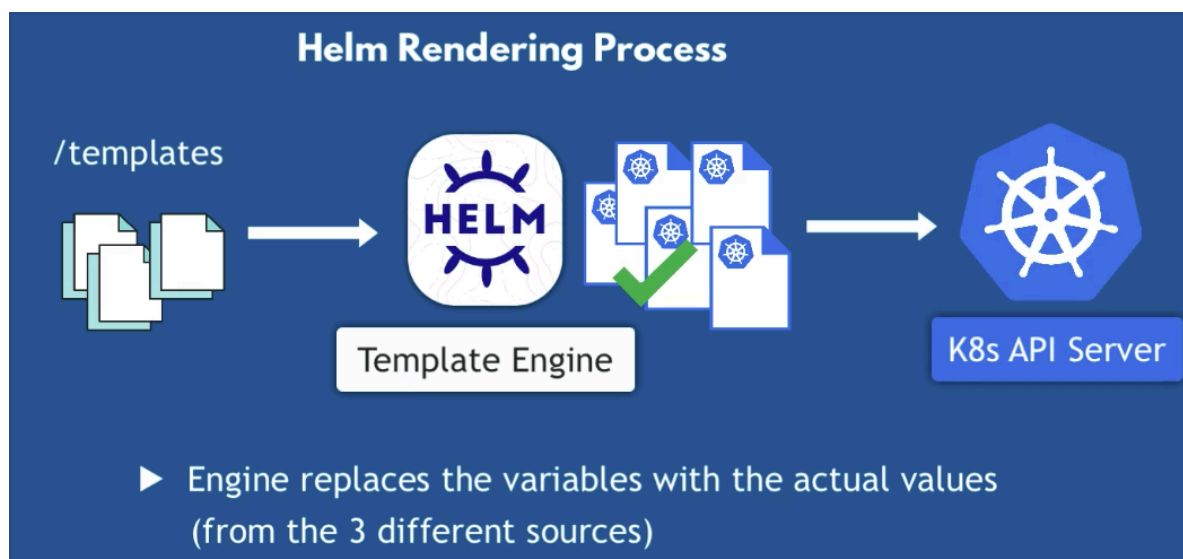
And as we can see we got a preview of the template filled with the actual values. We also could have use the below code:

```
-> helm template -f email-service-values.yaml --set appReplicas 3
microservice
```

The above looks at all the 3 values sources:

1. email-service-values.yaml -> source to get values
2. helm template -f email-service-values.yaml --set appReplicas 3
microservice -> is another source to get values
3. Using the default values.yaml file to get the default values that were not overridden like 'serviceType' placeholder var.

The process for -> helm install command



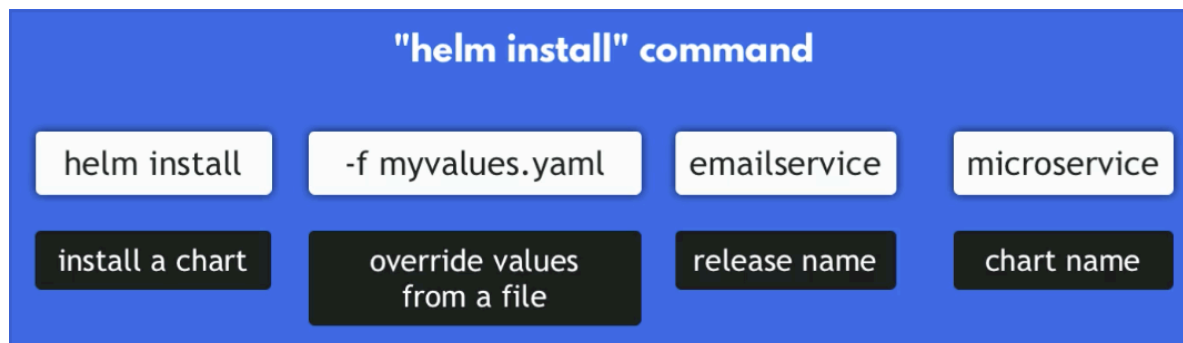
Another command is lint to check the correctness and validity of our yaml file:

-> helm lint -f email-service-values.yaml microservice

```
192:Module_10_chapter-21 astrid$ helm lint -f email-service-values.yaml microservice
==> Linting microservice
[INFO] Chart.yaml: icon is recommended

1 chart(s) linted, 0 chart(s) failed
```

Deploy EmailService using the helm chart



-> helm install -f email-service-values.yaml emailservice microservice

I had some error in the configuration of the templates so I got this error:

```
% helm install -f email-service-values.yaml emailservice microservice
Error: INSTALLATION FAILED: server-side apply failed for object default/emailservice apps/v1, Kind=Deployment: Deployment.apps "emailservice" is invalid: spec.template.spec.containers[0].env[0].name: Required value
```

I fixed it then I re-run -> helm install -f email-service-values.yaml emailservice microservice but the 'emailservice' release was already created when I run it the first time, so now I need to uninstall the release:

-> helm uninstall emailservice

And try again

-> helm install -f email-service-values.yaml emailservice microservice

And it worked 🥳 the chart is installed!

```
% helm install -f email-service-values.yaml emailservice microservice
NAME: emailservice
LAST DEPLOYED: Thu May 21 15:04:13 2026
NAMESPACE: default
STATUS: deployed
REVISION: 1
DESCRIPTION: Install complete
TEST SUITE: None
astrid@192.168.1.183 /Users/astrid/Module_10_chapter-21 [Module_10/Kubernetes_chapter-23]
```

-> helm ls

```
% helm ls
NAME          NAMESPACE    REVISION    UPDATED                               STATUS          CHART              APP VERSION
emailservice  default       1           2026-05-21 15:04:13.37443 +0100 BST  deployed        microservice-0.1.0 1.16.0
astrid@192.168.1.183 /Users/astrid/Module_10_chapter-21 [Module_10/Kubernetes_chapter-23]
```

We can see our emailservice microservice running as a chart.

-> kubectl get pod

```
% kubectl get pod
NAME                                READY   STATUS    RESTARTS   AGE
emailservice-778b6cfbdd-4n4rm      1/1     Running   0          4m38s
emailservice-778b6cfbdd-pnknq      1/1     Running   0          4m38s
astrid@192.168.1.183 /Users/astrid/Module_10_chapter-21 [Module_10/Kubernetes_chapter-23]
```

We can see the two replicas of my emailservice MS running in the cluster ★

Create values file for all microservices

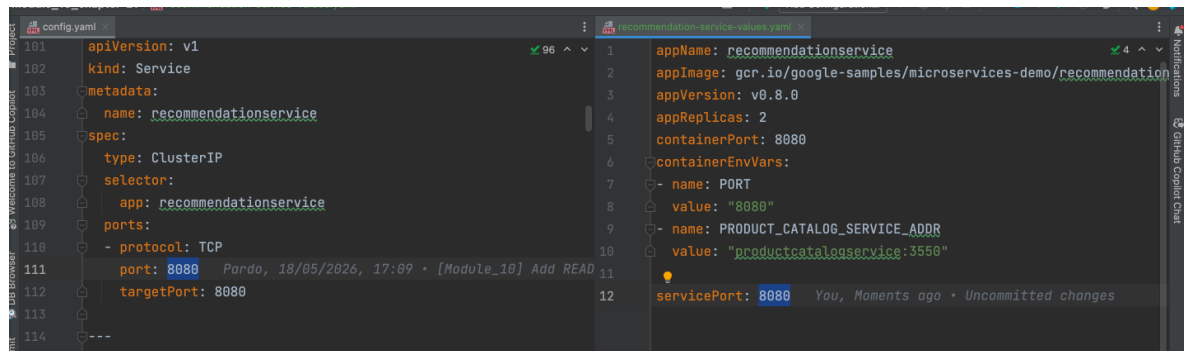
Let's do recommendationservice:

-copy the email-service-values.yaml and use it to create the new yaml

'recommendation-service-values.yaml'

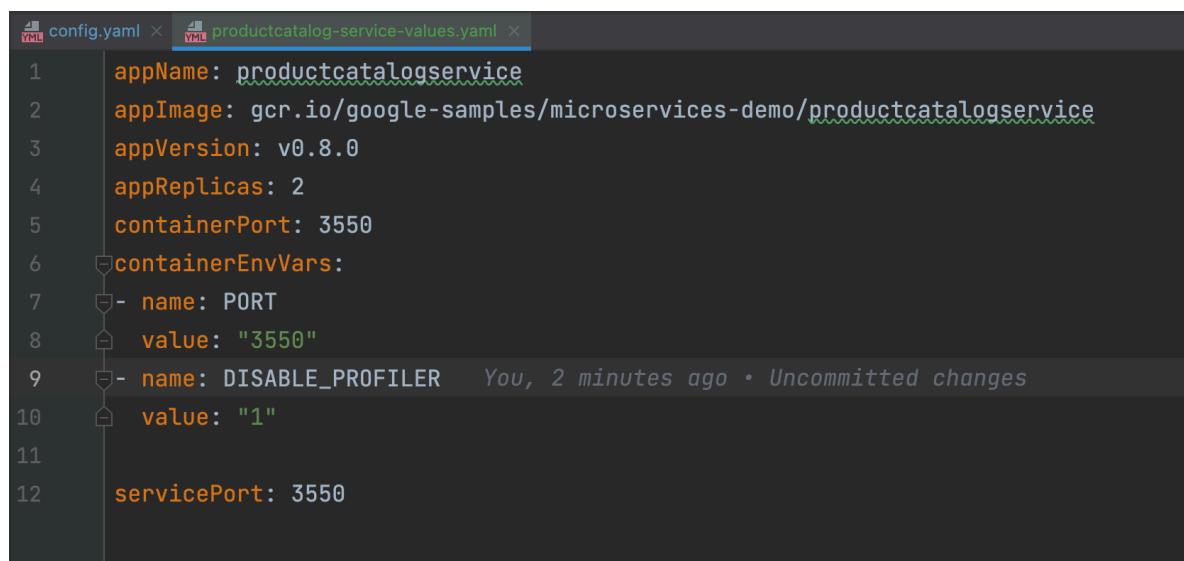
```
Module_10_chapter-21 > recommendation-service-values.yaml
1  appName: emailservice
2  appImage: gcr.io/google-samples/microservices-demo/emailservice
3  appVersion: v0.8.0
4  appReplicas: 2
5  containerPort: 8080
6  containerEnvVars: You, 50 minutes ago • Uncommitted changes
7  - name: PORT
8    value: "8080"
9
10 servicePort: 5000
```

And here it is:



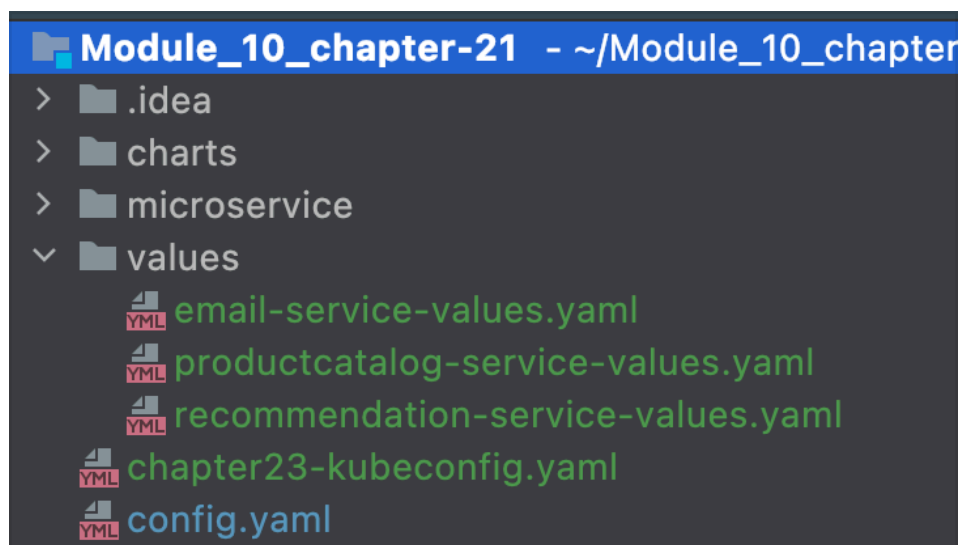
```
101 apiVersion: v1
102 kind: Service
103 metadata:
104   name: recommendationService
105 spec:
106   type: ClusterIP
107   selector:
108     app: recommendationService
109   ports:
110     - protocol: TCP
111       port: 8080
112       targetPort: 8080
113
114
1
2 appName: recommendationService
3 appImage: gcr.io/google-samples/microservices-demo/recommendation
4 appVersion: v0.8.0
5 appReplicas: 2
6 containerPort: 8080
7 containerEnvVars:
8   - name: PORT
9     value: "8080"
10   - name: PRODUCT_CATALOG_SERVICE_ADDR
11     value: "productcatalogservice:3550"
12 servicePort: 8080
```

Next productCatalogservice:

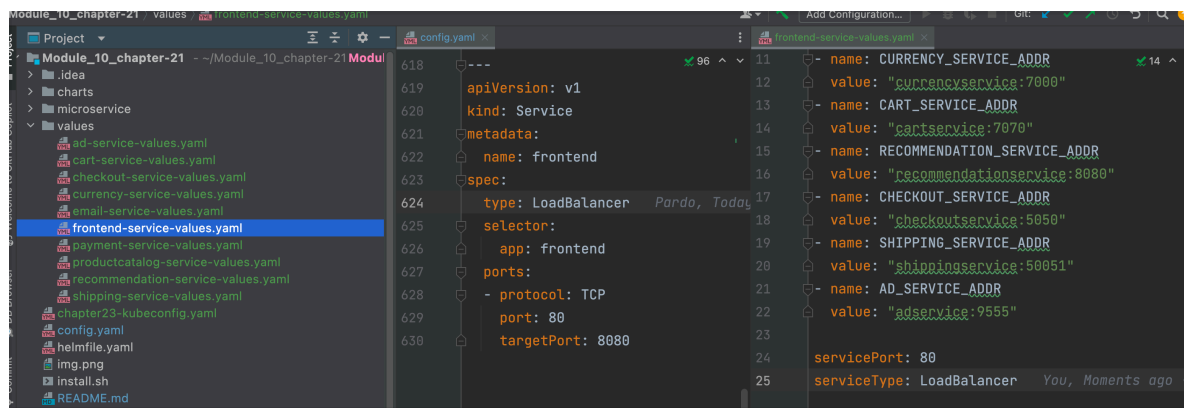


```
1 appName: productcatalogservice
2 appImage: gcr.io/google-samples/microservices-demo/productcatalogservice
3 appVersion: v0.8.0
4 appReplicas: 2
5 containerPort: 3550
6 containerEnvVars:
7   - name: PORT
8     value: "3550"
9   - name: DISABLE_PROFILER
10     value: "1"
11
12 servicePort: 3550
```

And I created a 'values' folder and moved the yamls there, as it is more organised:



Then I continue with the other MS (except redis service because it is a third party not one we developed ourselves, it has a different lifecycle):



Now let's install the recommendation service (remember -> it is now in a folder called 'values' and I need to use a new release name:

-> `helm install -f values/recommendation-service-values.yaml recommendationservice microservice`

```
astrid@192.168.1.183 /Users/astrid/Module_10_chapter-21 [Module_10/Kubernetes_chapter-23]
% helm install -f values/recommendation-service-values.yaml recommendationservice microservice
NAME: recommendationservice
LAST DEPLOYED: Thu May 21 15:38:23 2026
NAMESPACE: default
STATUS: deployed
REVISION: 1
DESCRIPTION: Install complete
TEST SUITE: None
astrid@192.168.1.183 /Users/astrid/Module_10_chapter-21 [Module_10/Kubernetes_chapter-23]
%
```

-> `helm ls`

```
astrid@192.168.1.183 /Users/astrid/Module_10_chapter-21 [Module_10/Kubernetes_chapter-23]
% helm ls
NAME                NAMESPACE    REVISION    UPDATED                               STATUS    CHART              APP VERSION
emailservice        default       1           2026-05-21 15:04:13.37443 +0100 BST  deployed  microservice-0.1.0  1.16.0
recommendationservice default       1           2026-05-21 15:38:23.791285 +0100 BST  deployed  microservice-0.1.0  1.16.0
astrid@192.168.1.183 /Users/astrid/Module_10_chapter-21 [Module_10/Kubernetes_chapter-23]
```

-> `kubectl get pod`

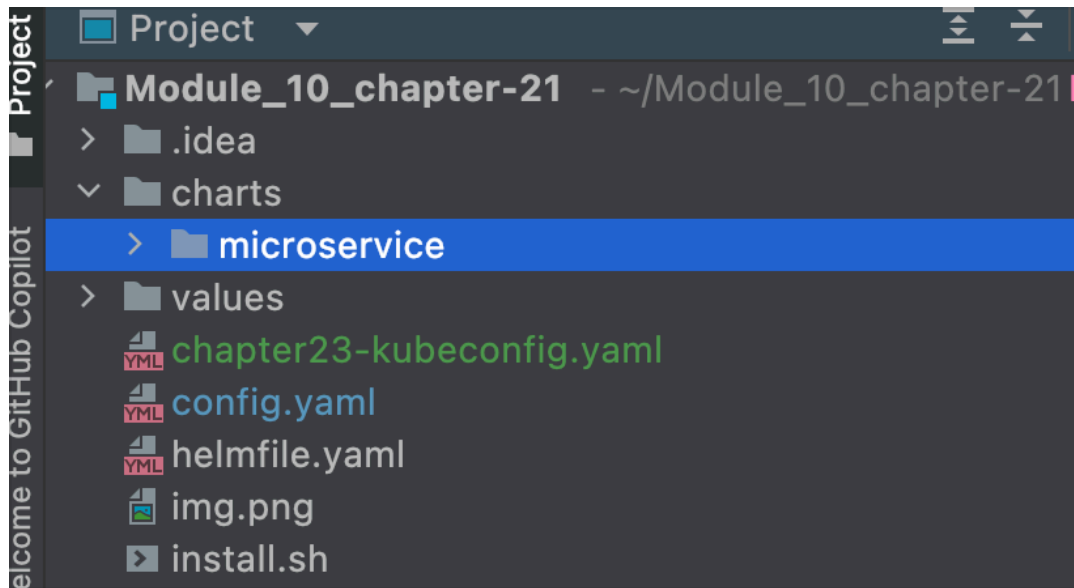
```
astrid@192.168.1.183 /Users/astrid/Module_10_chapter-21 [Module_10/Kubernetes_chapter-23]
% kubectl get pod
NAME                                READY    STATUS    RESTARTS    AGE
emailservice-778b6cfbdd-4n4rm       1/1      Running   0            35m
emailservice-778b6cfbdd-pnknq       1/1      Running   0            35m
recommendationservice-6b74f8cb9f-6hcnq 1/1      Running   0            73s
recommendationservice-6b74f8cb9f-pkbbx 1/1      Running   0            73s
astrid@192.168.1.183 /Users/astrid/Module_10_chapter-21 [Module_10/Kubernetes_chapter-23]
%
```

Create Redis Helm Chart

Now let's do redis-cart service, the third party app.

- We can use the configuration we already have for redis in the config.yaml file
- Or we can create the redis-cart own helm chart

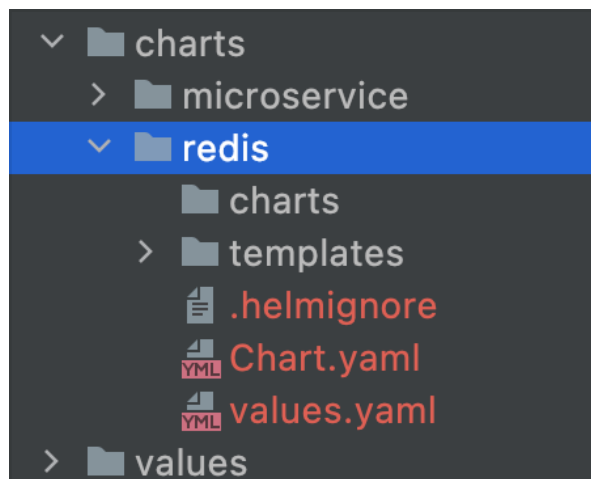
Nana created a new folder called 'charts' and moved 'microservice' folder there:



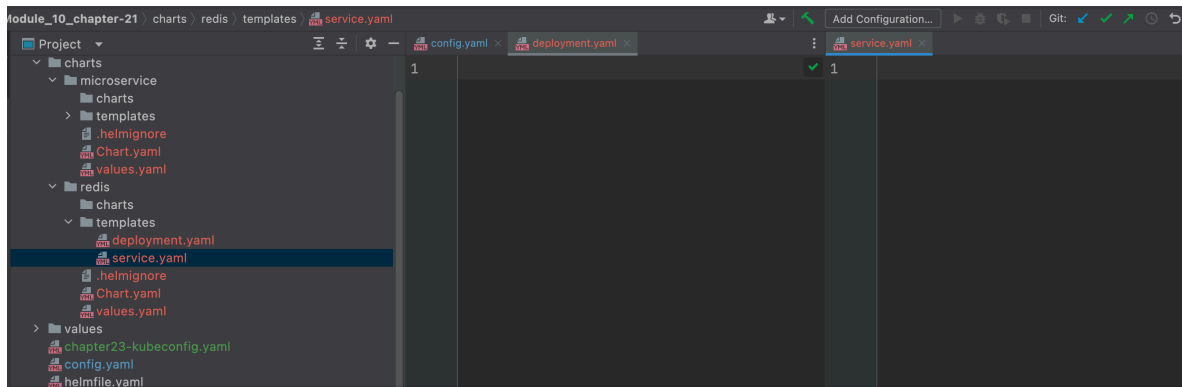
Now we will create a new chart inside the new folder, the same way we created the 'microservice' chart:

-> helm create redis

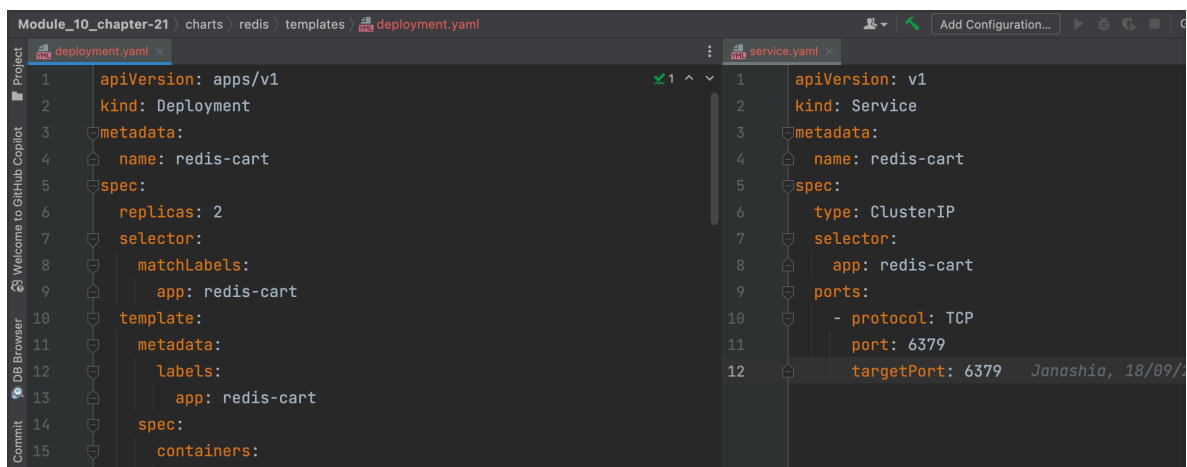
```
% helm create redis
Creating redis
```



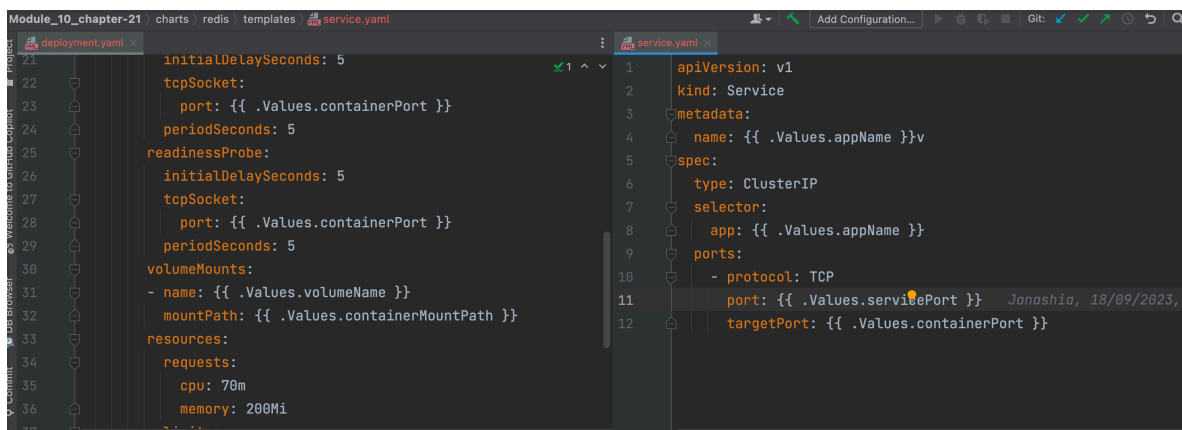
And remove files and content that I don't need:



Then I copied and pasted the configuration of redis-cart from the config.yaml to deployment.yaml and service.yaml:

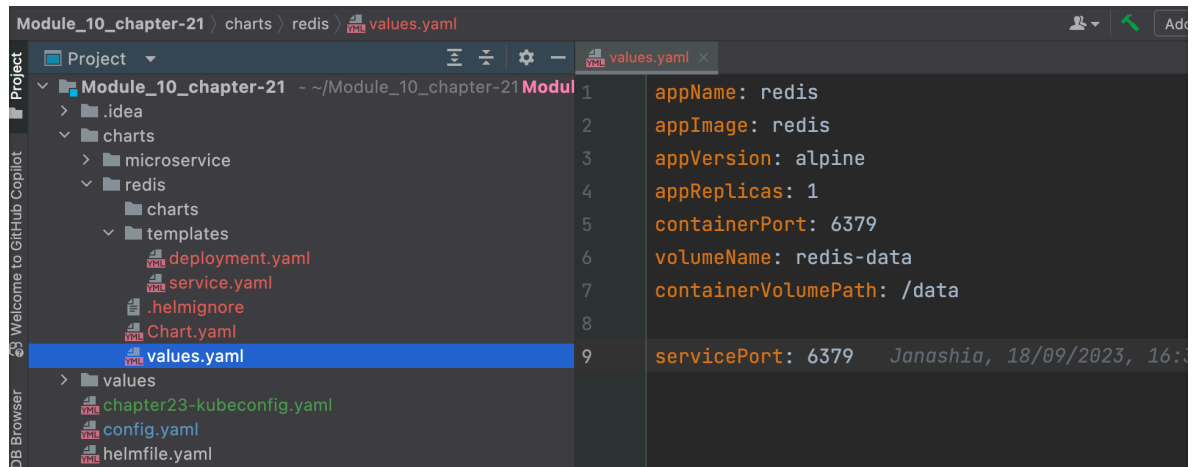


Now we will reverse engineer to transform the values into placeholder variables to create our template files:



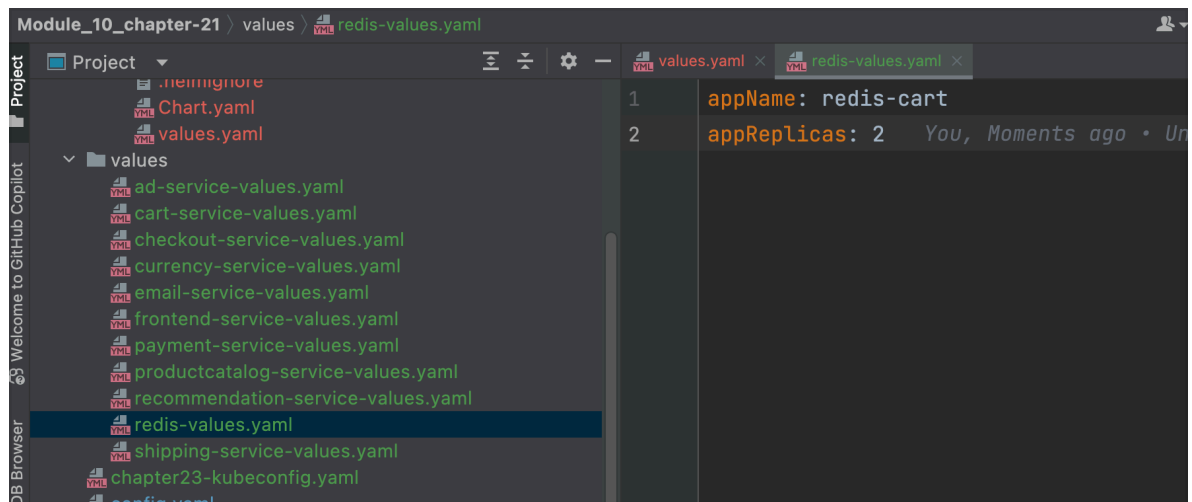
Most of the template values are parameterised.

Now let's define the values for redis:



```
Module_10_chapter-21 > charts > redis > values.yaml
1  appName: redis
2  appImage: redis
3  appVersion: alpine
4  appReplicas: 1
5  containerPort: 6379
6  volumeName: redis-data
7  containerVolumePath: /data
8
9  servicePort: 6379
```

Now let's define the actual values for redis name and its replicas to override the default values:



```
Module_10_chapter-21 > values > redis-values.yaml
1  appName: redis-cart
2  appReplicas: 2
```

And now let's validate our template:

-> `helm template -f values/redis-values.yaml charts/redis`

charts/redis -> 'redis' chart is in 'charts' folder

```

    app: redis-cart
  ports:
    - protocol: TCP
      port: 6379
      targetPort: 6379
---
# Source: redis/templates/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis-cart
spec:
  replicas: 2
  selector:
    matchLabels:
      app: redis-cart
  template:
    metadata:
      labels:
        app: redis-cart
    spec:
      containers:
        - name: redis-cart
          image: "redis:alpine"
          ports:
            - containerPort: 6379
          livenessProbe:
            initialDelaySeconds: 5
            tcpSocket:
              port: 6379
            periodSeconds: 5
          readinessProbe:
            initialDelaySeconds: 5
            tcpSocket:
              port: 6379
            periodSeconds: 5
          volumeMounts:
            - name: redis-data
              mountPath: /data
          resources:
            requests:
              cpu: 70m
              memory: 200Mi
            limits:
              cpu: 125m
              memory: 300Mi
      volumes:
        - name: redis-data
          emptyDir: {}

```

astrid@192.168.1.183 /Users/astrid/Module_10_chapter-21 [Module_10/Kubernetes_chapter-23]

Another way to preview the manifests is:

```
nana@macbook /Users/nana/k8s-configuration/microservices  
○ % helm install --dry-run -f values/redis-values.yaml charts/r
```

helm install --dry-run =
check generated manifest without installing the chart

Difference between --dry-run and template

- ▶ --dry-run send files to K8s cluster, while template only validates it locally



-> helm install --dry-run -f values/redis-values.yaml rediscart charts/redis

rediscart -> release name

```
astrid@192.168.1.183 /Users/astrid/Module_10_chapter-21 [Module_10/Kubernetes_chapter-23]
% helm install --dry-run -f values/redis-values.yaml rediscart charts/redis
level=WARN msg="--dry-run is deprecated and should be replaced with '--dry-run=client'"
NAME: rediscart
LAST DEPLOYED: Thu May 21 16:17:26 2026
NAMESPACE: default
STATUS: pending-install
REVISION: 1
DESCRIPTION: Dry run complete
TEST SUITE: None
HOOKS:
MANIFEST:
---
# Source: redis/templates/service.yaml
apiVersion: v1
kind: Service
metadata:
  name: redis-cartv
spec:
  type: ClusterIP
  selector:
    app: redis-cart
  ports:
    - protocol: TCP
      port: 6379
      targetPort: 6379
---
# Source: redis/templates/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis-cart
spec:
  replicas: 2
  selector:
    matchLabels:
      app: redis-cart
  template:
    metadata:
      labels:
        app: redis-cart
    spec:
      containers:
        - name: redis-cart
          image: "redis:alpine"
          ports:
            - containerPort: 6379
          livenessProbe:
```